

Warm up

Assuming 32-bit integers, which is larger: 0x00000000 or 0xFFFFFFFF?

- Depends on whether we interpret the values as signed or unsigned integers!
 - 0x00000000 is larger if interpreted as a signed integer (i.e., $0 > -1$)
 - 0xFFFFFFFF is larger if interpreted as an unsigned integer (i.e., $4294967295 > 0$)
- Follow-up: by just examining these two values, can we determine whether they are signed or unsigned?
 - No! Need additional context to determine how to interpret the values

Precept 18: ARMv8 control transfer, memory addressing modes and using gdb on assembly code

Polly Ren

COS 217: Introduction to Programming Systems

Spring 2026

Princeton University

April 7, 2026

Overview

- 1 Control transfer
- 2 Memory addressing modes
- 3 Using gdb to debug assembly code
- 4 Summary

Working with signedness

- We use **unsigned** integers in C when we want to represent only non-negative values, and **signed** integers when we want to represent both positive and negative values
- But, under the hood, all data is zeros and ones → how does the computer know whether to treat a value as signed or unsigned?
 - It doesn't! The same bits can be interpreted as either signed or unsigned, depending on the instruction used to manipulate it
 - ◇ whether a value is zero-extended (e.g., `ldrb`) or sign-extended (e.g., `ldrsh`) when loaded into a register
 - ◇ whether to perform division with signed (`sdiv`) or unsigned (`udiv`) values
 - ◇ today: how to perform control transfer

Control transfer with signed and unsigned values

- In C, we use relational operators (e.g., `<`, `>`, `==`) to compare values and control the flow of execution
- In ARMv8, we use branch instructions (e.g., `blt`, `bgt`, `beq`) to achieve the same effect
- Format of branch instructions: `b<cond> <symbol>`
 - `<cond>` specifies the condition under which to branch (e.g., `lt` for less than, `gt` for greater than, `eq` for equal)
 - `<symbol>` is the label to branch to if the condition is satisfied
- If the branch instruction does not itself compare values, what values are compared to determine whether to branch?
 - Use the condition flags in the `pstate` special-purpose register
 - Condition flags are set by certain instructions (e.g., `cmp`, `subs`) that perform comparisons or arithmetic operations
 - Example:

```
cmp w0, w1    // compare w0 and w1, set flags
blt label     // branch to label if w0 < w1
```

Condition flags

- For instructions that set condition flags, the flags are set based on the result of the operation
- The condition flags in the `pstate` register are:
 - Zero flag (Z): set if the result of an operation is zero
 - Negative flag (N): set if the result of an operation is negative (i.e., the most significant bit is 1)
 - Carry flag (C): set if an unsigned operation results in overflow (i.e., result as unsigned value is too large to fit in the register)
 - Overflow flag (V): set if a signed operation results in overflow (i.e., result as signed value is too large or too small to fit in the register)
- Branch instructions check these flags to determine whether to branch
- How are these flags set by `cmp`?
 - `cmp` is effectively a `subs` instruction: subtract second operand from first operand, set flags based on result, discard result
 - $a - b$ is really implemented as $a + (\sim b + 1)$
- Warning: other instructions can also set flags (e.g., `adds`, `subs`)
 - Implication: `cmp` is typically immediately followed by a branch

Aside: how flags get set

Assume registers are 8 bits for simplicity (i.e., -128 to 127 for signed integers, 0 to 255 for unsigned integers)

Operation	N	Z	C	V	Why
200 + 100	0	0	1	0	300 > 255 → unsigned overflow → C=1
127 + 1	1	0	0	1	128 > 127 → signed overflow → V=1 result = 0x80 → high bit set → N=1
200 + 200	1	0	1	0	400 > 255 → unsigned overflow → C=1 result = 0x90 → high bit set → N=1
1 + 1	0	0	0	0	$2 \leq 255$, $2 \leq 127$ → no overflow
1 + 255	0	1	1	0	256 > 255 → unsigned overflow → C=1 result = 0 → Z=1

Control transfer with signed and unsigned values

Instruction format: `b<cond> <symbol>`

cond	what it checks	condition flags to check
EQ	Equal	$Z==1$
NE	Not equal	$Z==0$
LT	Signed less than	$N!=V$
LE	Signed less than or equal	$N!=V \ \ Z==1$
GT	Signed greater than	$N==V \ \&\& \ Z==0$
GE	Signed greater than or equal	$N==V$
LO	Unsigned lower	$C==0$
LS	Unsigned lower or same	$C==0 \ \ Z==1$
HI	Unsigned higher	$C==1 \ \&\& \ Z==0$
HS	Unsigned higher or same	$C==1$

- Main takeaway: `cmp` is not sensitive to signedness, but the subsequent branch instruction is
 - Need not memorise the specifics of this table
- See *ARMv8 Condition Flags* handout for more details and conditions

Walkthrough of powerunsigned.{c, s}

- See `powerunsigned.c`, `powerunsignedflat.c` and `powerunsigned.s`; noteworthy aspects below
- `.quad 1`: allocate 8 bytes of memory for `ulPower` and initialise to 1
 - No notion of signedness at this point
- `cmp x0, x1`: compare `x0` and `x1`, set flags based on result of comparison
 - Flags are set based on the result of the subtraction `x0 - x1`
- `bhi powerLoopEnd`: branch to `powerLoopEnd` if there is a “high” relationship, i.e., if `x0 > x1` (as unsigned values)
 - Use `bhi` (unsigned) instead of `bgt` (signed)

Aside: bgt vs. bhi

- bgt: branch if greater than (signed comparison)
 - Branch if $N==V \ \&\& \ Z==0$
- bhi: branch if higher (unsigned comparison)
 - Branch if $C==1 \ \&\& \ Z==0$
- Example: say w0 contains 0xFFFFFFFF (i.e., -1 as a signed value, 4294967295 as an unsigned value) and w1 contains 0x0
 - `cmp w0, w1` → $N=1$ (result of $w0 - w1$ has high bit set), $Z=0$ (result is not zero), $C=1$ (unsigned overflow), $V=0$ (no signed overflow)
 - `bgt label` → does not branch because $N!=V$
 - `bhi label` → branches because $C==1 \ \&\& \ Z==0$

Overview

- 1 Control transfer
- 2 Memory addressing modes
- 3 Using gdb to debug assembly code
- 4 Summary

Memory addressing modes

- So far, we've only seen one way to specify the memory address for load and store instructions: register operands (e.g., `ldr w1, [x0]`)
 - Treat the value in the register as a memory address, load from / store to `reg[Xn]` (sort of like a pointer)
- ARMv8 supports several other addressing modes that allow us to specify memory operands in different ways
 - Can access data structures like arrays and structs more directly
 - Use fewer registers for intermediate values (e.g., index, field offset)
 - Can be more efficient (fewer instructions, fewer register reads / writes)
- *Immediate offset*: compute address as `reg[Xn] + imm`
 - `ldr w1, [x0, 12]` → load from address `reg[x0] + 12`
- *Register offset*: compute address as `reg[Xn] + reg[Xm]`
 - `ldr w2, [x0, x1]` → load from address `reg[x0] + reg[x1]`
- *Scaled register offset*: compute address as `reg[Xn] + (reg[Xm] << shift)`
 - `ldr w2, [x0, x1, lsl 2]` → load from address `reg[x0] + (reg[x1] << 2)`

Walkthrough: rev.c and assembly code

```
printf("%ld\n", alNums[lIndex])
```

rev.s

```
adr x0, printfFormatStr
adr x1, alNums
adr x2, lIndex
ldr x2, [x2]
lsl x2, x2, 3
add x1, x1, x2
ldr x1, [x1]
bl printf
```

revregoffset.s

```
adr x0, printfFormatStr
adr x1, alNums
adr x2, lIndex
ldr x2, [x2]
lsl x2, x2, 3
ldr x1, [x1, x2]
bl printf
```

revscaledregoffset.s

```
adr x0, printfFormatStr
adr x1, alNums
adr x2, lIndex
ldr x2, [x2]
ldr x1, [x1, x2, lsl 3]
bl printf
```

- Using register offset or scaled register offset addressing modes avoids the add instruction to compute address of `alNums[lIndex]`
- Note that shift amount in scaled register offset is determined by the size of the data type being accessed (e.g., 3 for `long` on armlab)
- Why can't we use register offset in the read loop?
 - Because `scanf` needs an address, not the value at the address
 - The write loop can use register offset because `printf` needs the value, which we can load using (scaled) register offset

Overview

- 1 Control transfer
- 2 Memory addressing modes
- 3 Using gdb to debug assembly code
- 4 Summary

Recommended steps for debugging assembly code

- Convert C code to flattened C code
- Build and test the flattened C code
- Check that each line of flattened C code exists in assembly code
 - Comments in the assembly code indicate which line of flattened C code corresponds to which assembly instructions
- Check that assembly code correctly implements the logic of flattened C code (i.e., that the translation from flattened C to assembly is correct)
 - Build and test the assembly code
- If errors persist, use gdb to step through the assembly code and inspect register values at each step
 - Similar to using gdb for C code, compile with -g flag
 - Can set breakpoints, step through instructions, print registers, etc.
 - See *GDB Tutorial for ARMv8 Assembly Language* handout

Overview

- 1 Control transfer
- 2 Memory addressing modes
- 3 Using gdb to debug assembly code
- 4 Summary

Summary

- How to use condition flags for control transfer with signed and unsigned values
- How condition flags are set by instructions like `cmp`
- Different memory addressing modes for load and store instructions
- Introduction to debugging assembly code with `gdb`
- Dates to remember:
 - Assignment 4: due Tue, 4/7 at 9pm (tonight!)
 - Assignment 4 Quiz: Mon, 4/13 in lecture (next week, not tomorrow)
- Acknowledgements
 - Most of the code in these slides is adapted from the precept materials, written by Prof. Bob Dondero and William Ughetta
 - Can be accessed at <https://www.cs.princeton.edu/courses/archive/spring26/cos217/precepts/w11p1/>